

MLP + Batch Normalization

Batch Normalization on top of the neural language model — Kaiming initialisation, the three-step BN transform, running statistics, tanh saturation, and the deeper network. After Andrej Karpathy's makemore.

The plain MLP works, but as we stack layers the activations drift: some neurons saturate **tanh**, gradients shrink, and training stalls. **Batch Normalization** (Ioffe & Szegedy, 2015) fixes this by re-centring each layer's pre-activations to mean 0 and variance 1 across the mini-batch, then letting the network rescale them with two learned parameters. This document derives every formula in `build_makemore_batchnorm.ipynb`.

What stays the same as the plain MLP

The objective is unchanged: embed the context, run it through a hidden layer, produce logits, apply **softmax**, minimise average **cross-entropy**, and learn by backprop. BatchNorm only changes *how the pre-activations are conditioned* before the non-linearity — and adds a clean way to do inference one example at a time.

1. The forward pipeline

With BatchNorm inserted, one mini-batch of size $B = 32$ flows through six stages before the loss. The new block sits **between the linear layer and the tanh**:

$$C[X] \rightarrow \text{concat} \rightarrow XW_1 \rightarrow \text{BN}(\gamma, \beta) \rightarrow \tanh \rightarrow hW_2 + b_2 \rightarrow \mathcal{L}$$

Stage	Operation	Shape (B=32)	Note
Embedding	$C[Xb]$	(32, 3, 10)	look up 3 chars × 10-dim
Concat	<code>emb.view(B, -1)</code>	(32, 30)	flatten context
Linear W1	<code>embcat @ W1</code>	(32, 200)	no bias (BN follows)
BatchNorm	$\gamma \cdot \text{xhat} + \beta$	(32, 200)	normalise + scale/shift
Tanh	<code>tanh(hBN)</code>	(32, 200)	values in (-1, +1)
Linear W2	<code>h @ W2 + b2</code>	(32, 27)	27 = vocab size
Loss	<code>cross_entropy</code>	scalar	avg negative log-prob

2. Embeddings: each character becomes a vector

Each character index i selects one row of a learned table $\mathbf{C}$ of shape (27, 10). The three context characters give a (3, 10) block, flattened to a 30-vector:

$$\text{emb} = [C[c_1], C[c_2], C[c_3]] \in \mathbb{R}^{3 \times 10} \Rightarrow \text{embcat} \in \mathbb{R}^{30}$$

Looking up a row equals multiplying a one-hot vector by C — a one-hot row selects exactly one row of C — so indexing is just the fast path. Dense vectors let the model place **similar characters near each other**, the generalisation a sparse count table cannot get.

3. Linear layer W_1 and Kaiming initialisation

The first linear layer maps the 30-dim embedding to 200 hidden pre-activations. It carries **no bias** — BatchNorm immediately subtracts the batch mean, so any additive bias would cancel, and BN's own β replaces it:

$$h_{\text{pre}} = \text{embcat} \cdot W_1, \quad W_1 \in \mathbb{R}^{30 \times 200}, \quad \text{no bias}$$

If $w \sim \mathcal{N}(0,1)$ and the input has unit variance, the output variance is $n \cdot \text{Var}(w)$. To keep it near 1 we scale the weights — and for tanh we add the empirical gain 5/3:

$$W_1 \sim \mathcal{N}\left(0, \left(\frac{5/3}{\sqrt{n_{\text{in}}}}\right)^2\right), \quad n_{\text{in}} = 30$$

```
W1 = torch.randn((30, 200), generator=g) * (5/3) / (30**0.5)
```

Why the 5/3?

A plain $1/\sqrt{n}$ (Xavier) slightly under-drives tanh; raw randn massively over-drives it ($\text{std} \approx 5.5 \rightarrow$ most units saturate before training even starts). The 5/3 gain is tuned so that, after tanh's mild squashing, the activations keep unit variance.

Initialisation	std(h_pre)	tanh saturation	gradient health
raw randn (no scale)	~5.5	~85%	very weak
Xavier $1/\sqrt{n}$	~1.0	~28%	moderate
Kaiming-tanh $(5/3)/\sqrt{n}$	~1.67	~5%	strong

4. Batch Normalization: the three-step transform

BN is applied per feature, across the B examples of the mini-batch. **Step 1** — compute the mean and variance of each neuron over the batch:

$$\mu_j = \frac{1}{m} \sum_{i=1}^m h_{\text{pre}, i, j}, \quad \sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (h_{\text{pre}, i, j} - \mu_j)^2$$

Step 2 — normalise each value to zero mean, unit variance (ϵ guards against division by zero):

$$\hat{h}_{i,j} = \frac{h_{\text{pre}, i, j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}, \quad \epsilon = 10^{-5}$$

Step 3 — scale and shift with two **learnable** parameters γ (bngain) and β (bnbias), one per neuron:

$$h_{\text{BN}, i, j} = \gamma_j \hat{h}_{i, j} + \beta_j$$

Why undo the normalisation you just did?

Forcing every layer to stay exactly mean 0, variance 1 is too rigid — some features need a different scale or offset. γ and β give that freedom back while keeping gradients well-conditioned. At init $\gamma=1$, $\beta=0$, so BN starts as the identity on the normalised values: the safest possible starting point.

5. The BN parameters γ and β

Tensor	Symbol	Init	Role
bngain	γ	1.0	scale the normalised activations
bnbias	β	0.0	shift the normalised activations
bnmean_running	μ_{run}	0.0	global mean used at inference
bnstd_running	σ_{run}	1.0	global std used at inference

γ and β are trained by gradient descent like any weight; their gradients are simple sums over the batch:

$$\frac{\partial \mathcal{L}}{\partial \gamma_j} = \sum_i \delta_{i,j} \hat{h}_{i,j}, \quad \frac{\partial \mathcal{L}}{\partial \beta_j} = \sum_i \delta_{i,j}$$

μ_{run} and σ_{run} are **not** learned — they are bookkeeping, updated by a moving average (next section).

6. Running statistics: training vs. inference

During training BN uses the current batch's μ and σ . But at inference you may score one character at a time, and a batch of size 1 has no usable variance. The fix: keep an **exponential moving average** of the statistics during training, then use it at test time:

$$\mu_{\text{run}} \leftarrow (1 - \alpha) \mu_{\text{run}} + \alpha \mu_{\text{batch}}, \quad \sigma_{\text{run}} \leftarrow (1 - \alpha) \sigma_{\text{run}} + \alpha \sigma_{\text{batch}}$$

```
with torch.no_grad():
    bnmean_running = 0.999 * bnmean_running + 0.001 * bnmean_i
    bnstd_running = 0.999 * bnstd_running + 0.001 * bnstd_i
```

The train → inference bridge

$\alpha = 0.001$ in the notebook's inline code (very stable); the `BatchNorm1d` class uses $\alpha = 0.1$. Switching the whole network over is one flag per layer: `layer.training = False`, which branches BN from the batch-stats path to the running-stats path. This EMA is the piece that is unique to BatchNorm.

7. Tanh and gradient saturation

The normalised, rescaled pre-activations pass through `tanh`. Its derivative is largest at 0 and collapses toward the tails — this is exactly what BN protects against:

$$h = \tanh(h_{\text{BN}}), \quad \frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$$

input x	$\tanh(x)$	$1 - \tanh^2(x)$	gradient health
0.0	0.000	1.000	maximum
1.0	0.762	0.420	moderate
2.0	0.964	0.071	weak — saturating
3.0	0.995	0.010	near zero
5.0	0.9999	0.0002	effectively dead

The notebook flags a neuron as saturated when $|\tanh(x)| > 0.97$, i.e. $|x| > 2.1$, where less than 6% of the gradient survives. Because BN keeps `h_BN` centred near 0, saturation stays rare even after stacking many layers — the whole reason BN lets deep tanh networks train.

Without BatchNorm

After random init the activation distribution drifts layer by layer; a large fraction of neurons saturate, gradients vanish, and the deep network fails to learn. BN re-centres every layer's input to the responsive zone of `tanh`, layer after layer.

8. Output layer and cross-entropy loss

A second linear layer maps the 200 hidden features to 27 logits; softmax turns them into probabilities and the loss is the average negative log-likelihood of the true next character:

$$\text{logits} = h W_2 + b_2, \quad p_j = \frac{e^{\text{logit}_j}}{\sum_k e^{\text{logit}_k}}, \quad \mathcal{L} = -\frac{1}{N} \sum_i \log p_{i, y_i}$$

W_2 is initialised tiny ($\times 0.01$) so all 27 logits start near zero → near-uniform softmax → initial loss $\approx -\log(1/27) \approx 3.30$. Starting from maximum uncertainty avoids wasting the first steps unlearning a confidently-wrong init. (In the deep model the same idea is `layers[-1].gamma *= 0.1`.)

9. Backpropagation through the stack

Loss → logits is the usual softmax gradient; the interesting part is BatchNorm. Through the affine and normalise steps:

$$\frac{\partial \mathcal{L}}{\partial \gamma} = \sum_i \delta_i \hat{h}_i, \quad \frac{\partial \mathcal{L}}{\partial \beta} = \sum_i \delta_i$$
$$\frac{\partial \mathcal{L}}{\partial h_{\text{pre},i}} = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} \left(\delta_i - \frac{1}{m} \sum_k \delta_k - \frac{\hat{h}_i}{m} \sum_k \delta_k \hat{h}_k \right)$$

The coupling that makes BN special

Each sample's input gradient depends on *all* other samples in the batch — through the two sum terms above. This is why BN stabilises training but also why a tiny batch (B=1) breaks it: one sample cannot estimate a useful mean and variance.

10. Parameter update and training loop

Every parameter — including γ and β — is updated by minibatch SGD with a decayed learning rate:

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$

```
for i in range(max_steps):
    ix = torch.randint(0, Xtr.shape[0], (32,))      # minibatch
    # forward: embed -> W1 -> BatchNorm -> tanh -> W2 -> loss
    loss.backward()
    lr = 0.1 if i < 100000 else 0.01                # decay
    for p in parameters: p.data += -lr * p.grad
```

What BatchNorm buys you

It makes training robust to the exact initialisation and lets you stack many layers without activations exploding or vanishing. The deeper 6-layer network (47,024 params) trains smoothly to a loss near 2.0 — and crucially, the architecture now scales.

11. A worked numerical example

Take four pre-activations of one neuron across a batch, $h_{\text{pre}} = [5.0, 1.0, -3.0, 2.0]$, with $\gamma = 1.5$, $\beta = 0.5$:

Quantity	Computation	Value
μ	$(5 + 1 - 3 + 2) / 4$	1.25
σ^2	mean of squared deviations	8.19
σ	$\sqrt{8.19}$	2.86
$\hat{x}$ (i=0)	$(5.0 - 1.25) / 2.86$	+1.31

$\hat{x}$ (i=2)	$(-3.0 - 1.25) / 2.86$	-1.49
h_{BN} (i=0)	$1.5 \times 1.31 + 0.5$	+2.47
h (i=0)	$\tanh(2.47)$	0.986
$\partial L / \partial \gamma$	$\sum \delta \cdot \hat{x}$ (given δ)	+0.314
$\partial L / \partial \beta$	$\sum \delta$	+0.126

12. Custom classes and the deeper network

The notebook 'PyTorchifies' the code into reusable modules, then stacks them into a 6-layer network:

Class	Mimics	Parameters	Forward
Linear	nn.Linear	weight, (bias)	$x @ \text{weight} + \text{bias}$
BatchNorm1d	nn.BatchNorm1d	gamma, beta	$\gamma \cdot (x - \mu) / \sigma + \beta$
Tanh	nn.Tanh	—	$\tanh(x)$

Each hidden block is `Linear(bias=False) → BatchNorm1d → Tanh`, repeated five times, then an output block — about **47,024** parameters in total. The final layer uses `layers[-1].gamma *= 0.1` to keep the first logits small.

Linear vs Linear + BN (the std=31 experiment)

Pushing 1000 unit-Gaussian inputs through a 1000-wide linear layer blows the output std up to $\sqrt{1000} \approx 31$, and the same factor appears in the backward pass. Inserting BN pins the forward output to $\text{std} \approx 1$ and stabilises the input gradients to ≈ 1 — catching the blow-up in both directions.

Loss progression across the notebook's fixes:

Change	train	val
original MLP baseline	2.125	2.168
fix softmax init ($W_2 \times 0.01$)	2.07	2.13
fix tanh saturation (W_1 init)	2.036	2.103
+ Batch Normalization	2.067	2.105

BatchNorm's headline numbers are not dramatically lower here — the win is **robustness and scalability**: training no longer hinges on a perfectly tuned initialisation, and deep stacks finally converge.